

Conception et développement d'IHM

Cours S3: Maîtriser les bases de React

Introduction et prérequis: contenu

- La notion de composant dans React
- La syntaxe JSX
- Bibliothèques de composants UI
- Transmettre des propriétés (props) à un composant
- Réagir aux événements
- Mise à jour de l'écran: state
- Partage des données entre les composants
- Exemple récapitulatif de communication inter-composants

La notion de composant dans React

L'HTML nous permet de créer des documents structurés riches grâce à ses balises telles que `<h1>` et `<li>`:

```
<article>
  <h1>My First Component</h1>
  <ol>
    <li>Components: UI Building Blocks</li>
    <li>Defining a Component</li>
    <li>Using a Component</li>
  </ol>
</article>
```

React vous permet de combiner vos balises, CSS et JavaScript dans des "composants" personnalisés, des éléments d'interface utilisateur réutilisables pour votre application. Le code de la table des matières que vous avez vu ci-dessus pourrait être transformé en un composant `<TableOfContents />` que vous pourriez afficher sur chaque page. Sous le capot, il utilise toujours les mêmes balises HTML comme `<article>`, `<h1>`, etc.

```
<PageLayout>
  <NavigationHeader>
    <SearchBar />
    <Link to="/docs">Docs</Link>
  </NavigationHeader>
  <Sidebar />
  <PageContent>
    <TableOfContents />
    <DocumentationText />
  </PageContent>
</PageLayout>
```

La notion de composant dans React

Un composant React est une simple fonction qui retourne du JSX. Le nom d'un composant doit commencer par une majuscule.

Les déclarations de retour peuvent être écrites sur une seule ligne, comme dans ce composant :

```
return ;
```

Mais si votre balisage ne se trouve pas sur la même ligne que le mot-clé de retour, vous devez l'entourer d'une paire de parenthèses, sans parenthèses, tout code figurant sur les lignes suivant le retour sera ignoré.

```
return (  
  <div>  
      
  </div>  
);
```

Les composants sont des fonctions JavaScript normales, de sorte que vous pouvez conserver plusieurs composants dans le même fichier. C'est pratique lorsque les composants sont relativement petits ou étroitement liés les uns aux autres. Si ce fichier devient encombré, vous pouvez toujours déplacer un composant dans un fichier séparé.

Introduction à JSX

Le web s'est construit sur HTML, CSS et JavaScript. Pendant de nombreuses années, les développeurs web ont conservé le contenu en HTML, la conception en CSS et la logique en JavaScript - souvent dans des fichiers séparés ! Le contenu était balisé en HTML, tandis que la logique de la page vivait séparément en JavaScript :

```
<div>
  <p></p>
  <form>
  </form>
</div>
```

HTML

```
isLoggedIn() {...}
onClick() {...}
onSubmit() {...}
```

JavaScript

A mesure que le Web devenait plus interactif, la logique déterminait de plus en plus le contenu. JavaScript était en charge du HTML ! C'est pourquoi , dans React, la logique de rendu et le balisage cohabitent au même endroit - les composants.

```
Sidebar() {
  if (isLoggedIn()) {
    <p>Welcome</p>
  } else {
    <Form />
  }
}
```

Sidebar.js React component

```
Form() {
  onClick() {...}
  onSubmit() {...}

  <form onSubmit>
    <input onClick />
    <input onClick />
  </form>
}
```

Form.js React component

Les règles de JSX

1. Renvoyer un seul élément racine. Pour renvoyer plusieurs éléments d'un composant, il suffit de les entourer d'une seule balise mère, ou entre la balise vide `<>` et `</>`
2. Fermer toutes les balise: JSX exige que les balises soient explicitement fermées : les balises qui se ferment d'elles-mêmes comme `<img>` doivent devenir `<img />`, et les balises enveloppantes comme `<li>oranges` doivent être écrites comme `<li>oranges</li>`
3. Les attributs HTML doivent respecter la syntaxe camelCase. Par exemple, au lieu de `stroke-width`, on utilise `strokeWidth`. Comme `class` est un mot réservé, dans JSX on écrit `className` à la place.

JSX: utilisation du JavaScript à l'intérieur de JSX

JSX est une façon particulière d'écrire JavaScript. Cela signifie qu'il est possible d'utiliser JavaScript à l'intérieur - avec des accolades { }. L'exemple ci-dessous commence par déclarer un nom pour une personne, name, puis l'incorpore avec des accolades à l'intérieur de l'élément <h1>.

```
export default function TodoList() {  
  const name = 'ALAMI Ahmed';  
  return (  
    <h1>{name}'s To Do List</h1>  
  );  
}
```

N'importe quelle expression JavaScript fonctionnera entre accolades. Outre les chaînes de caractères, les nombres et d'autres expressions JavaScript, vous pouvez également transmettre des objets en JSX. Les objets sont également désignés par des accolades, comme { name : "ALAMI", inventions : 5 }. Par conséquent, pour passer un objet JS en JSX, vous devez envelopper l'objet dans une autre paire d'accolades : person={{ name : "ALAMI", inventions : 5 }}. Vous pouvez voir cela dans les styles CSS en ligne de JSX.

NB:

Les propriétés de style en ligne sont écrites en camelCase. Par exemple, HTML <ul style="background-color : black"> serait écrit comme <ul style={{ backgroundColor : 'black' }}> dans votre composant.

Transmettre des propriétés (props) à un composant

Les composants React utilisent des propriétés (props) pour communiquer entre eux. Chaque composant parent peut transmettre des informations à ses composants enfants en leur donnant des propriétés. Les « props » ressemblent aux attributs HTML, mais vous pouvez leur passer n'importe quelle valeur JavaScript, y compris des objets, des tableaux et des fonctions.

Transmettre des propriétés à un composant enfant:

Passons deux propriétés: person (un objet), et size (un nombre)

```
export default function Profile() {  
  return (  
    <Avatar  
      person={{ name: 'ALAMI Ahmed', imageId: '1bX5QH6' }}  
      size={100}  
    />  
  );  
}
```

Récupération des propriétés dans le composant fils:

Vous pouvez lire ces propriétés en listant leurs noms: person, size séparée par les virgules à l'intérieur ({ et }) directement après la fonction Avatar. Cela vous permet de les utiliser à l'intérieur du code Avatar, comme vous le feriez avec une variable.

```
function Avatar({ person, size }) {  
  // person et size sont disponibles ici  
}
```

Transmettre des composants JSX en tant qu'enfants

Parfois, c'est utile d'imbriquer les composants JSX:

```
<Card>
  <Avatar />
</Card>
```

Lorsque vous imbriquez du contenu dans une balise JSX, le composant parent reçoit ce contenu dans une propriété appelée **children**. Par exemple, le composant Card ci-dessous recevra une propriété children définie sur <Avatar /> et l'affichera dans une div enveloppante :

```
import Avatar from './Avatar.js';

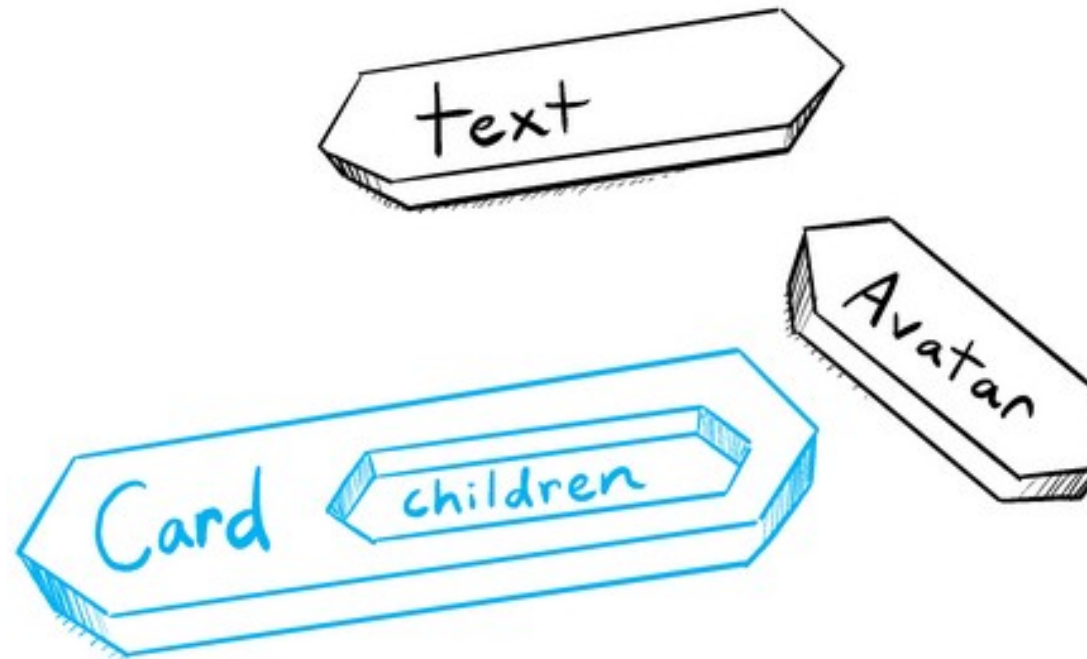
function Card({ children }) {
  return (
    <div className="card">
      {children}
    </div>
  );
}

export default function Profile() {
  return (
    <Card>
      <Avatar
        size={100}
        person={{ name: 'ALAMI Ahmed', imageId: 'Yfe0qp2' }}
      />
    </Card>
  );
}
```

Transmettre des composants JSX en tant qu'enfants

Essayez de remplacer `<Avatar>` à l'intérieur de `<Card>` par du texte pour voir comment le composant Card peut envelopper tout contenu imbriqué. Il n'a pas besoin de "savoir" ce qui est mis à l'intérieur. Ce « design pattern » est très utilisé dans React.

On peut considérer qu'un composant doté d'une propriété enfant possède un "trou" qui peut être "comblé" par ses composants parents à l'aide d'un JSX arbitraire. Vous utiliserez souvent la propriété `children` pour les enveloppes visuelles : panneaux, grilles, etc.



Rendu conditionnel

Dans React, il n'y a pas de syntaxe spéciale pour écrire des conditions. Au lieu de cela, vous utiliserez les mêmes techniques que lorsque vous écrivez du code JavaScript normal. Par exemple, vous pouvez utiliser une instruction if pour inclure JSX de manière conditionnelle :

```
let content;
if (isLoggedIn) {
  content = <AdminPanel />;
} else {
  content = <LoginForm />;
}
return (
  <div>
    {content}
  </div>
);
```

Ou avec l'opérateur ternaire:

```
<div>
  { isLoggedIn ? <AdminPanel /> : <LoginForm /> }
</div>
```

Affichage des listes

Le boucle for et la fonction `Array.map()` sont utilisées pour afficher les listes. Par exemple, supposons que vous ayez un tableau de produits :

```
const products = [ { title: 'Cabbage', id: 1 }, { title: 'Garlic', id: 2 }, { title: 'Apple', id: 3 }, ];
```

Dans votre composant, utilisez la fonction `map()` pour transformer un tableau de produits en un tableau d'éléments `<li>` :

```
const listItems = products.map(product =>
  <li key={product.id}>
    {product.title}
  </li>
);

return (
  <ul>{listItems}</ul>
);
```

Remarquez que `<li>` possède un attribut `key`. Pour chaque élément d'une liste, vous devez fournir une chaîne de caractères ou un nombre qui identifie de manière unique cet élément parmi ses frères et sœurs. En général, une clé doit provenir de vos données, comme l'identifiant d'une base de données. React utilise vos clés pour savoir ce qui s'est passé si vous insérez, supprimez ou réorganisez les éléments par la suite.

Préserver la pureté des composants

Les fonctions pures n'effectuent qu'un calcul et rien d'autre. En n'écrivant strictement les composants en tant que fonctions pures, vous pouvez éviter toute une série de bogues et de comportements imprévisibles.

React part du principe que chaque composant que vous écrivez est une fonction pure. Cela signifie que les composants React que vous écrivez doivent toujours renvoyer le même JSX avec les mêmes entrées, et ne pas modifier les objets ou les variables qui existaient avant l'affichage - ce qui les rendrait impurs !

```
let guest = 0;

function Cup() {
  guest = guest + 1;
  return <h2>Tea cup for guest #{guest}</h2>;
}

export default function TeaSet() {
  return (
    <>
      <Cup />
      <Cup />
    </>
  );
}
```

Ce composant lit et écrit une variable déclarée à l'extérieur. Cela signifie qu'appeler ce composant plusieurs fois produira des JSX différents ! De plus, si d'autres composants lisent cette variable, ils produiront également des JSX différents.

Les bonnes pratiques des composants JSX

La solution à ce problème est de passer « guest » en tant que « prop »:

```
function Cup({ guest }) {  
  return <h2>Tea cup for guest #{guest}</h2>;  
}  
  
export default function TeaSet() {  
  return (  
    <>  
      <Cup guest={1} />  
      <Cup guest={2} />  
      <Cup guest={3} />  
    </>  
  );  
}
```

Ce composant est maintenant pur, car le JSX qu'il renvoie ne dépend que de la propriété « guest »

Bibliothèques de composants UI

Les bibliothèques de composants UI sont des collections d'éléments d'interface utilisateur prêts à l'emploi ou précodés qui permettent d'accélérer le développement de logiciels. Ces éléments préconçus vont des boutons, des fenêtres modales et des cartes à des composants complexes tels que des tableaux de données et des graphiques.

Les bibliothèques de composants UI sont créées pour simplifier la création d'interfaces utilisateur en fournissant aux développeurs des blocs de construction et des éléments qui respectent les meilleures pratiques en matière de conception et d'accessibilité.

Les avantages de l'utilisation d'une bibliothèque de composants UI:

- **Rapidité et productivité:** l'utilisation d'une bibliothèque de composants UI simplifie et accélère le processus de design et développement.
- **Facile à utiliser :** L'apprentissage du CSS prend du temps et la conception d'interfaces utilisateur complexes peut s'avérer difficile. Avec les bibliothèques UI, même les débutants peuvent importer et utiliser les éléments disponibles sans trop se concentrer sur le CSS.
- **Cohérence:** Les bibliothèques UI garantissent la cohérence de l'aspect et de la convivialité de votre application, car elles sont conçues à l'aide d'un guide de style cohérent.
- **Personnalisation:** Bien que ces bibliothèques offrent des composants prêts à l'emploi, elles sont souvent hautement personnalisables pour répondre aux exigences spécifiques de votre projet.
- **Accessibilité:** De nombreuses bibliothèques d'interface utilisateur donnent la priorité à l'accessibilité, ce qui facilite la création d'applications utilisables par tous.

Les meilleures bibliothèques de composants UI en 2024

Ci-dessous la liste des meilleures bibliothèques de composants UI React en 2024. Cette liste n'inclut que les bibliothèques open source et gratuites.

- [Material UI](#): c'est la bibliothèque UI la plus populaire dans l'écosystème React. Elle suit le style « Material Design » de Google. Elle a plus de 89k étoiles GitHub et plus de 3 millions de téléchargements hebdomadaires sur NPM.
- [Chakra UI](#): Il s'agit d'une autre bibliothèque React UI populaire et accessible. Elle a plus de 34,6k étoiles sur GitHub et 480k téléchargements hebdomadaires sur NPM
- [Ant Design](#): une bibliothèque React UI bien documentée avec des composants React de haute qualité. Avec plus de 87k étoiles sur GitHub et plus d'un million de téléchargements hebdomadaires sur NPM.
- [Mantine](#): une bibliothèque React UI qui dispose de plus de 100 composants personnalisables et de 50 « hooks ». Elle a plus de 21k étoiles GitHub et plus de 250k téléchargements hebdomadaires.
- [Shadcn UI](#): est une nouvelle bibliothèque de composants React sortie en Mars 2023, elle est maintenant en pleine expansion. Elle a plus de 55k étoiles GitHub et plus de 56k téléchargements hebdomadaires sur NPM. Les composants de shadcn ne sont pas seulement prêts à l'emploi, mais aussi prêts à être modifiés. En fait, avec shadcn, le code des composants va être copié dans votre projet, ce qui n'est pas le cas des autres bibliothèques. Donc vous allez avoir le contrôle total sur le code par la suite.

En plus des bibliothèques UI, il y a aussi des templates React installable qui facilite et accélère d'avantage le développement, comme par exemple [Material Kit React](#)

Réagir aux événements

On peut répondre aux événements en déclarant des fonctions de gestion d'événements dans les composants :

```
function MyButton() {  
  function handleClick() {  
    alert('You clicked me!');  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Click me  
    </button>  
  );  
}
```

Remarquez que `onClick={handleClick}` n'a pas de parenthèses à la fin ! N'appellez pas la fonction de gestion de l'événement : vous devez seulement la transmettre. React appellera votre gestionnaire d'événement lorsque l'utilisateur cliquera sur le bouton.

Vous pouvez également définir un gestionnaire d'événements en même ligne:

```
<button onClick={() => alert('You clicked me!'); }>
```

Transmettre les gestionnaires d'événements en tant que propriétés

Souvent, vous voudrez que le composant parent spécifie le gestionnaire d'événement d'un enfant. Prenons l'exemple des boutons : selon l'endroit où vous utilisez un composant Bouton, vous pouvez vouloir exécuter une fonction différente - peut-être que l'une d'entre elles lit une vidéo et qu'une autre télécharge une image.

```
function Button({ onClick, children }) {
  return (
    <button onClick={onClick}>
      {children}
    </button>
  );
}

function PlayButton({ movieName }) {
  function handlePlayClick() { alert(`Playing ${movieName}!`); }

  return (
    <Button onClick={handlePlayClick}>
      Play "{movieName}"
    </Button>
  );
}

function UploadButton() {
  return (
    <Button onClick={() => alert('Uploading!')}>
      Upload Image
    </Button>
  );
}
```

Mise à jour de l'écran: states

Souvent, vous voudrez que votre composant se "souviene" de certaines informations et les affiche. Par exemple, vous voudrez peut-être compter le nombre de fois qu'un bouton est cliqué. Pour se faire, ajoutez un état (State) à votre composant.

Tout d'abord, importez **useState** de React :

```
import { useState } from 'react';
```

Vous pouvez maintenant déclarer une variable d'état dans votre composant :

```
function MyButton() {  
  const [count, setCount] = useState(0);  
  // ...  
}
```

Vous obtiendrez deux choses de `useState` : l'état actuel (`count`) et la fonction qui vous permet de le mettre à jour (`setCount`). Vous pouvez leur donner n'importe quel nom, mais la convention est d'écrire [`something`, `setSomething`].

Mise à jour de l'écran

La première fois que le bouton est affiché, le compte sera de 0 car vous avez passé 0 à `useState()`. Lorsque vous souhaitez changer d'état, appelez `setCount()` et passez-lui la nouvelle valeur. Cliquer sur ce bouton incrémentera le compteur :

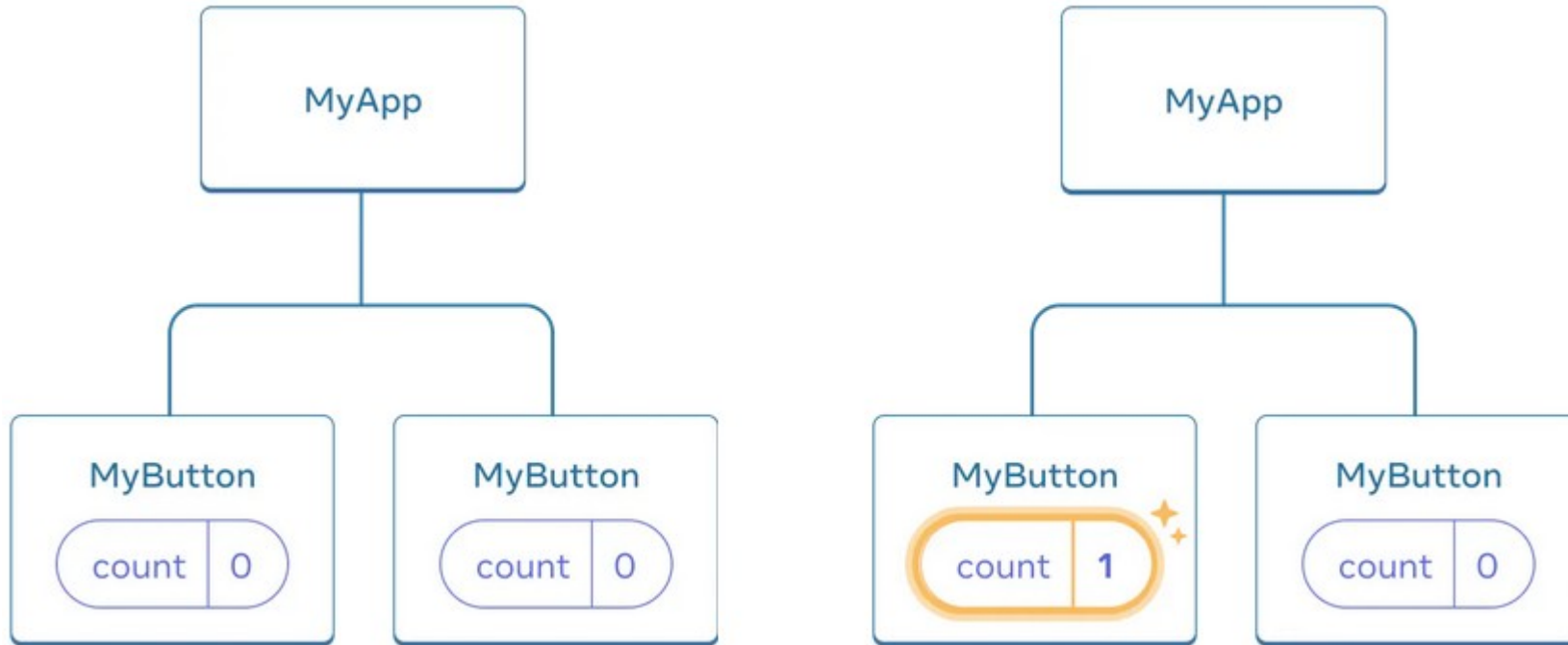
```
function MyButton() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    setCount(count + 1);  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Clicked {count} times  
    </button>  
  );  
}
```

React appellera à nouveau la fonction de votre composant. Cette fois, le compte sera de 1. Puis ce sera 2. Et ainsi de suite.

Si vous affichez le même composant plusieurs fois, chacun aura son propre état.

Partage des données entre les composants

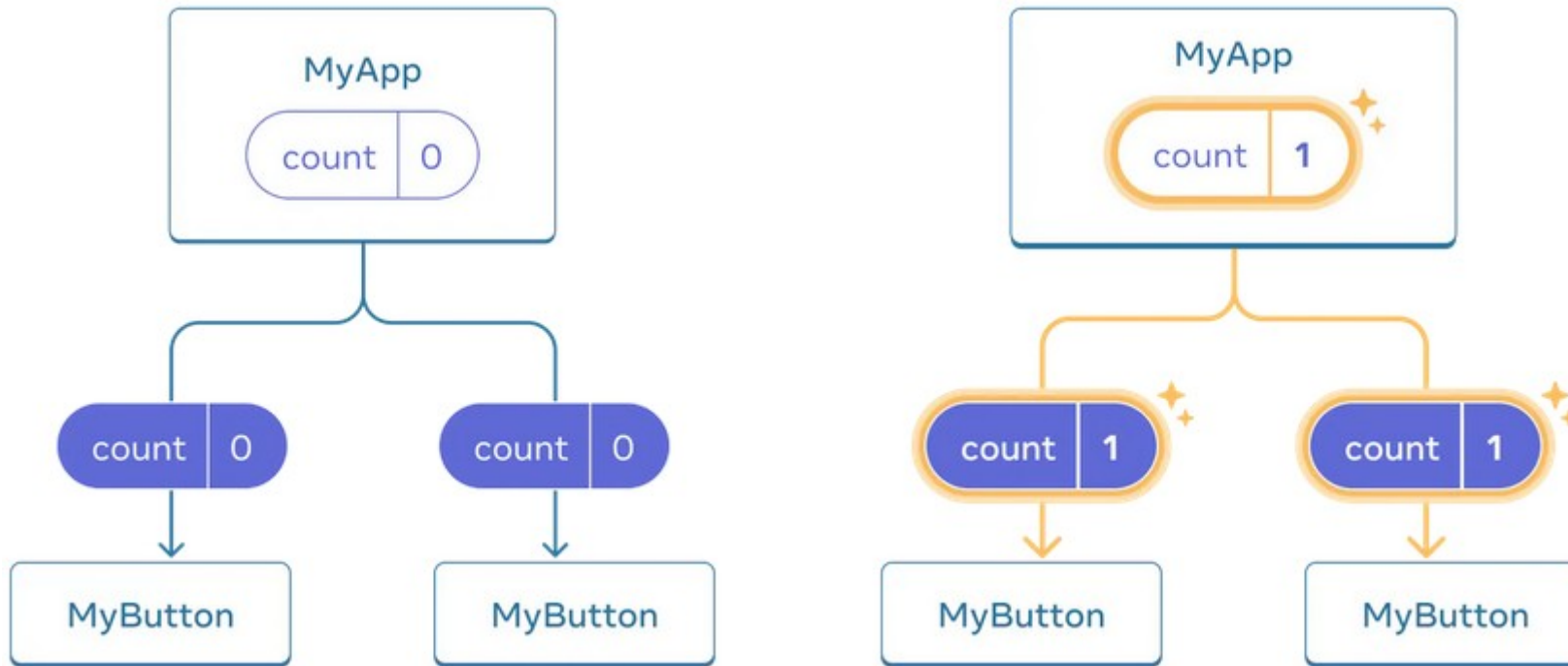
Dans le slide précédent, chaque composant MyButton avait son propre état (state) indépendant, et lorsque l'on cliquait sur chaque bouton, seul l'état du bouton cliqué changeait :



Cependant, vous aurez souvent besoin de composants qui partagent des données et qui sont toujours mis à jour ensemble.

Partage des données entre les composants

Pour que les deux composants MyButton affichent le même nombre et se mettent à jour ensemble, vous devez déplacer l'état (state) des boutons individuels "vers le haut", vers le composant le plus proche qui les contient tout les deux.



Désormais, lorsque vous cliquez sur l'un ou l'autre des boutons, l'état de MyApp change, ce qui modifie les deux états de MyButton.

Partage des données entre les composants

Voici comment vous pouvez exprimer cela en code. Tout d'abord, déplacez l'état de MyButton vers MyApp :

```
export default function MyApp() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <h1>Counters that update separately</h1>
      <MyButton />
      <MyButton />
    </div>
  );
}

function MyButton() {
  // ... nous déplaçons le code d'ici et nous le mettons dans MyApp ...
}
```

Partage des données entre les composants

Ensuite, transmettez l'état de MyApp à chaque MyButton, ainsi que le gestionnaire de clic partagé. Vous pouvez transmettre des informations à MyButton en utilisant les accolades JSX.

```
export default function MyApp() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <h1>Compteurs qui se mettent à jour ensemble</h1>
      <MyButton count={count} onClick={handleClick} />
      <MyButton count={count} onClick={handleClick} />
    </div>
  );
}
```

Les informations que vous transmettez de cette manière sont appelées "**props**". Le composant MyApp contient l'état de comptage et le gestionnaire d'événement handleClick, et les transmet tous deux à chacun des boutons en tant que **props**.

Partage des données entre les composants

Enfin, modifiez MyButton pour qu'il lise les propriétés que vous lui avez transmis depuis son composant parent :

```
function MyButton({ count, onClick }) {  
  return (  
    <button onClick={onClick}>  
      Clicked {count} times  
    </button>  
  );  
}
```

Lorsque vous cliquez sur le bouton, le gestionnaire onClick se déclenche. La propriété onClick de chaque bouton a été définie sur la fonction handleClick de MyApp, de sorte que le code qu'elle contient s'exécute. Ce code appelle setCount(count + 1), qui incrémente la variable d'état count. La nouvelle valeur de comptage est transmise à chaque bouton en tant que propriété, de sorte qu'ils affichent tous la nouvelle valeur. C'est ce qu'on appelle "élever l'état" (lifting state up). En déplaçant l'état vers le haut, vous l'avez partagé entre les composants.

« Render and Commit »

Avant que vos composants ne soient affichés à l'écran, ils doivent être rendus par React. La procédure de demande et d'affichage de des interfaces se déroule en trois étapes :

1. Étape 1 : Déclencher un rendu:

Il y a deux raisons pour lesquelles un composant doit être actualisé:

- a. Il s'agit du rendu initial du composant.
- b. L'état « state » du composant (ou de l'un de ses ancêtres) a été mis à jour.

2. Étape 2 : React affiche vos composants

Après avoir déclenché un rendu, React appelle vos composants pour déterminer ce qu'il faut afficher à l'écran. Le "rendu" est l'appel par React de vos composants.

- Lors du premier rendu, React appellera le composant racine.
- Pour les rendus suivants, React appellera le composant de fonction dont la mise à jour de l'état a déclenché le rendu.

3. Étape 3 : React apporte les modifications au DOM

Après avoir rendu (appelé) vos composants, React va modifier le DOM.

- Pour le rendu initial, React utilisera l'API DOM `appendChild()` pour mettre tous les nœuds DOM qu'il a créés à l'écran.
- Pour les rendus ultérieurs, React appliquera les opérations minimales nécessaires (calculées pendant le rendu) pour que le DOM corresponde à la dernière sortie du rendu.

Mise à jour des objets dans l'état « state »

L'état « state » peut contenir n'importe quel type de valeur JavaScript, y compris des objets. Mais vous ne devez pas modifier directement les objets que vous détenez dans l'état React. Au lieu de cela, lorsque vous souhaitez mettre à jour un objet, vous devez en créer un nouveau (ou faire une copie d'un objet existant), puis configurer l'état pour utiliser cette copie.

Considérer un objet dans l'état « state »:

```
const [position, setPosition] = useState({ x: 0, y: 0 });
```

Techniquement, il est possible de modifier le contenu de l'objet lui-même. C'est ce qu'on appelle une mutation :

```
position.x = 5;
```

Cependant, bien que les objets dans React state soient techniquement mutables, vous devriez les traiter comme s'ils étaient immuables, comme les nombres, les booléens et les chaînes de caractères. Au lieu de les modifier, vous devez toujours les remplacer. En d'autres termes, vous devez considérer tout objet JavaScript que vous mettez en état comme étant en lecture seule.

Mise à jour des objets dans l'état « state »

Cet exemple maintient un objet dans l'état pour représenter la position actuelle d'un point.

```
import { useState } from 'react';
export default function MovingDot() {
  const [position, setPosition] = useState({
    x: 0,
    y: 0
  });
  return (
    <div
      onPointerMove={e => {
        position.x = e.clientX;
        position.y = e.clientY;
      }}
    </div>
  );
}
```

Ce code modifie l'objet assigné à la position du rendu précédent. Mais sans utiliser la fonction `setPosition`, React n'a aucune idée que l'objet a changé. Pour déclencher un nouveau rendu dans ce cas, créez un nouvel objet et transmettez-le à la fonction de définition de l'état:

```
onPointerMove={e => {
  setPosition({
    x: e.clientX,
    y: e.clientY
  });
}}
```

Mise à jour des tableaux dans l'état « state »

Les tableaux sont mutables en JavaScript, mais vous devez les considérer comme immuables lorsque vous les stockez dans l'état. Comme pour les objets, lorsque vous souhaitez mettre à jour un tableau stocké dans l'état, vous devez en créer un nouveau (ou faire une copie d'un tableau existant), puis paramétrer l'état pour qu'il utilise le nouveau tableau.

En JavaScript, les tableaux sont un autre type d'objet. Comme pour les objets, vous devez traiter les tableaux dans React state en lecture seule. Cela signifie que vous ne devez pas réaffecter des éléments à l'intérieur d'un tableau comme `arr[0] = 'bird'`, et vous ne devez pas non plus utiliser des méthodes qui modifient le tableau, comme `push()` et `pop()`

Au lieu de cela, chaque fois que vous voudrez mettre à jour un tableau, vous devrez passer un nouveau tableau à votre fonction de définition de l'état. Pour ce faire, vous pouvez créer un nouveau tableau à partir du tableau original dans votre état en appelant ses méthodes de non-mutation comme `filter()` et `map()`. Ensuite, vous pouvez passer votre état au nouveau tableau résultant.

Mise à jour des tableaux dans l'état « state »

Voici un tableau récapitulatif des opérations courantes sur les tableaux. Lorsque vous manipulez des tableaux dans React state, vous devez éviter les méthodes de la colonne de gauche et utiliser plutôt celles de la colonne de droite:

	À éviter (modifie le tableau)	Préférable (renvoie un nouveau tableau)
Ajout	push, unshift	concat, [...arr] syntaxe « spread »
Suppression	pop, shift, splice	filter, slice
Remplacement	splice, arr[i] = ... affectation	map
Trie	reverse, sort	copier d'abord le tableau

Mise à jour des tableaux dans l'état « state »: exemple d'ajout

```
...  
const [name, setName] = useState('');  
const [artists, setArtists] = useState([]);  
let nextId = 0;  
...  
<button onClick={() => {  
  artists.push({  
    id: nextId++,  
    name: name,  
  });  
}}>Add</button>  
...
```

```
setArtists(  
  [  
    ...artists,  
    { id: nextId++, name: name }  
  ]  
);
```

Le code à gauche modifie le tableau. Au lieu de cela, le code à droite crée un nouveau tableau qui contient les éléments existants et un nouvel élément à la fin. Il existe plusieurs façons de faire ceci, mais la plus simple est d'utiliser la syntaxe « spread » de tableau (...)

La syntaxe d'étalement « spread » des tableaux vous permet également d'ajouter un élément en le plaçant avant le tableau (...artists) d'origine. De cette façon, « spread » peut faire le travail de push() en ajoutant à la fin d'un tableau et de unshift() en ajoutant au début d'un tableau.

Mise à jour des tableaux dans l'état « state »: exemple de suppression et transformation

La façon la plus simple de supprimer un élément d'un tableau est de le filtrer. En d'autres termes, vous produirez un nouveau tableau qui ne contiendra pas cet élément. Pour ce faire, utilisez la méthode **filter**, par exemple :

```
setArtists(  
  artists.filter(a => a.id !== artist.id)  
);
```

Transformer un tableau:

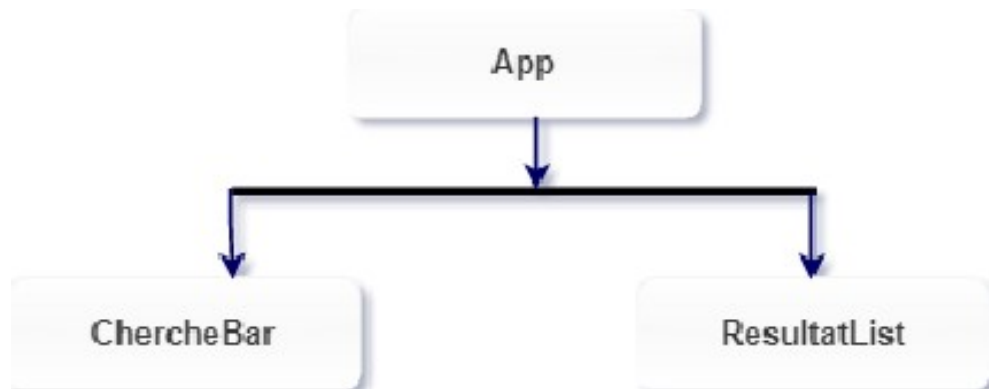
Si vous souhaitez modifier tout ou partie des éléments du tableau, vous pouvez utiliser la fonction **map** pour créer un nouveau tableau. La fonction que vous passerez à map peut décider de ce qu'il faut faire avec chaque élément, en fonction de ses données ou de son index (ou des deux).

Exemple récapitulatif de communication inter-composants

Dans cet exemple, nous disposons de trois composants :

- App
- ChercherBar
- ResultatList

Les composants ChercherBar et ResultatList sont imbriqués dans le composant App



Composant App

composant ChercheBar

Entrer le mot clé de recherche:

le type :LEGUME

composant ResultatList

- tomate
- carotte
- pomme de terre
- navet
- poivron

Exemple récapitulatif de communication inter-composants

On dispose de la liste ci-dessous:

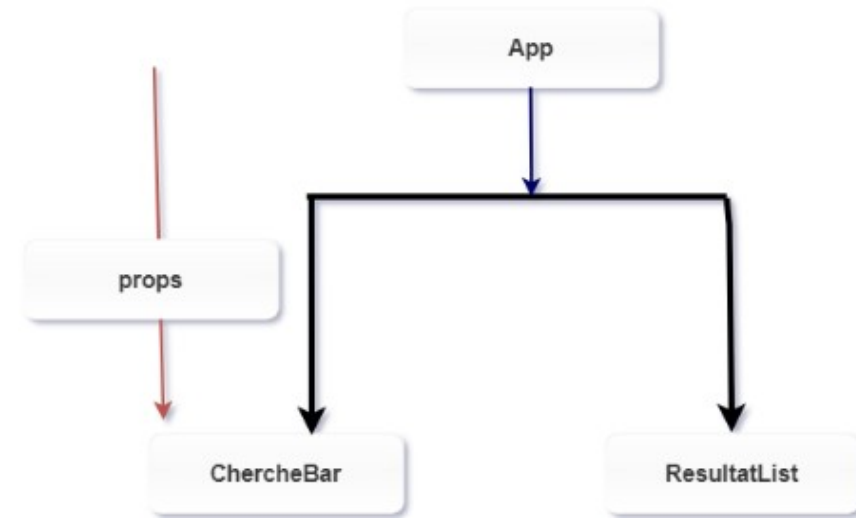
```
const list=[
  {nom:"banane",type:"fruit"},
  {nom:"orange",type:"fruit"},
  {nom:"pomme",type:"fruit"},
  {nom:"raisins",type:"fruit"},
  {nom:"kiwi",type:"fruit"},
  {nom:"tomate",type:"legume"},
  {nom:"carotte",type:"legume"},
  {nom:"pomme de terre",type:"legume"},
  {nom:"navet",type:"legume"},
  {nom:"poivron",type:"legume"}
]
```

Cette liste est une constante dans le composant App. L'utilisateur saisi le type dans le composant ChercheBar, la soumission du formulaire déclenche un callback de la fonction qui aura la valeur saisie dans Cherchebar comme argument et qui va filtrer la liste selon le type saisie, le composant ResultatList affichera par la suite les éléments filtrés.

Exemple de communication inter-composants: communication du parent vers l'enfant

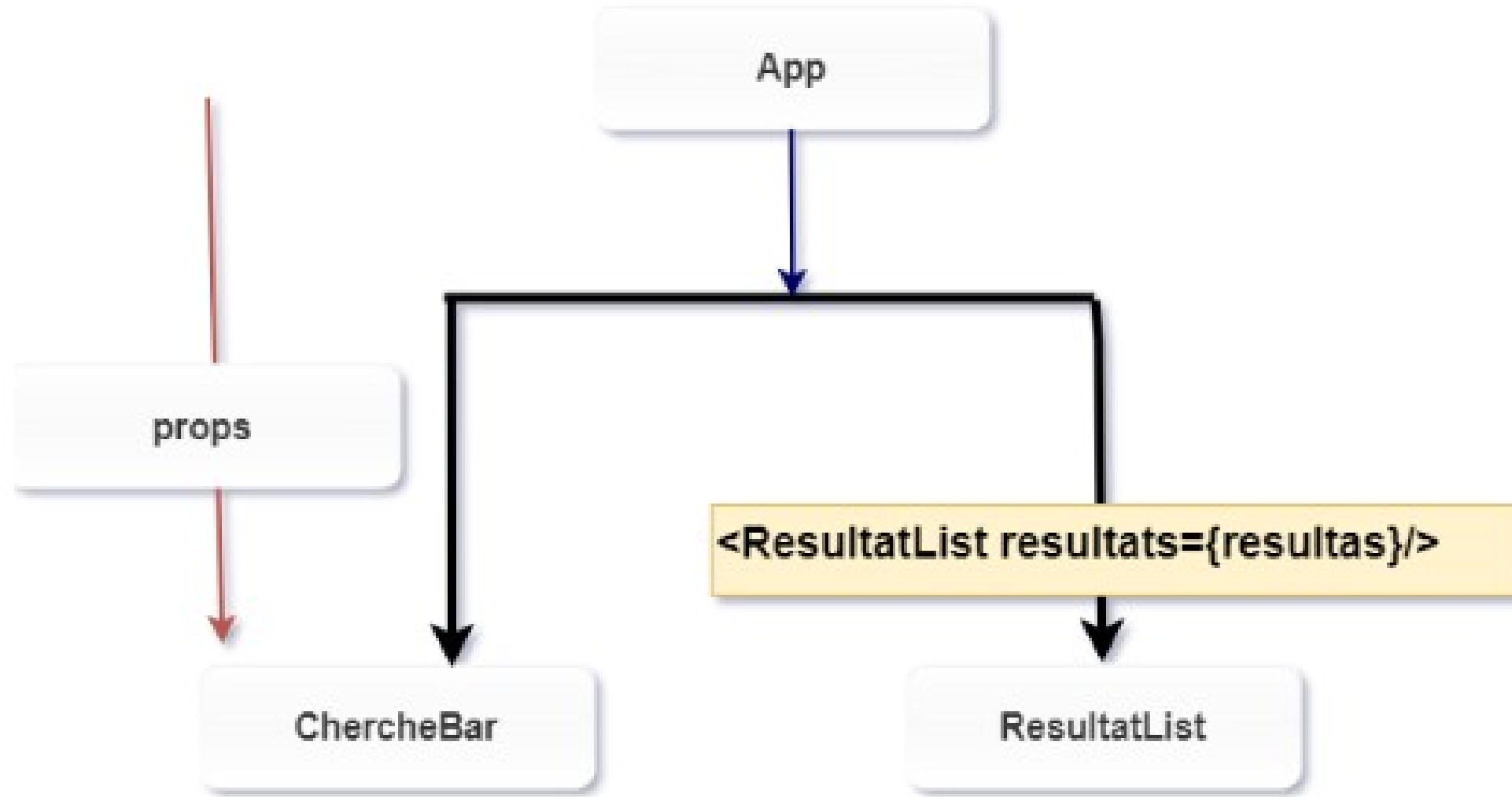
App.js:

```
import { useState } from 'react' ;
import ChercheBar from './components/ChercheBar';
import ResultatList from './components/ResultatList';
const list=[
  {nom:"banane",type:"fruit"},
  ...]
export default function App() {
  const [type,setType]=useState('')
  const [resultas,setResultas]=useState([])
  function onChercheSubmitApp(type){
    setType(type)
    setResultas(list.filter(item => item.type.toUpperCase()==type))
  }
  return (
    <div className='App'>
      <h1>Composant App</h1>
      <ChercheBar onChercheSubmit={onChercheSubmitApp}/>
      <div>
        <p>le type:<span
style={{color:"rgb(36,44,33)",fontWeight:"bold"}}>{type}</span></p>
        </div>
        <ResultatList resultats={resultas}/>
      </div>
    );
  }
```

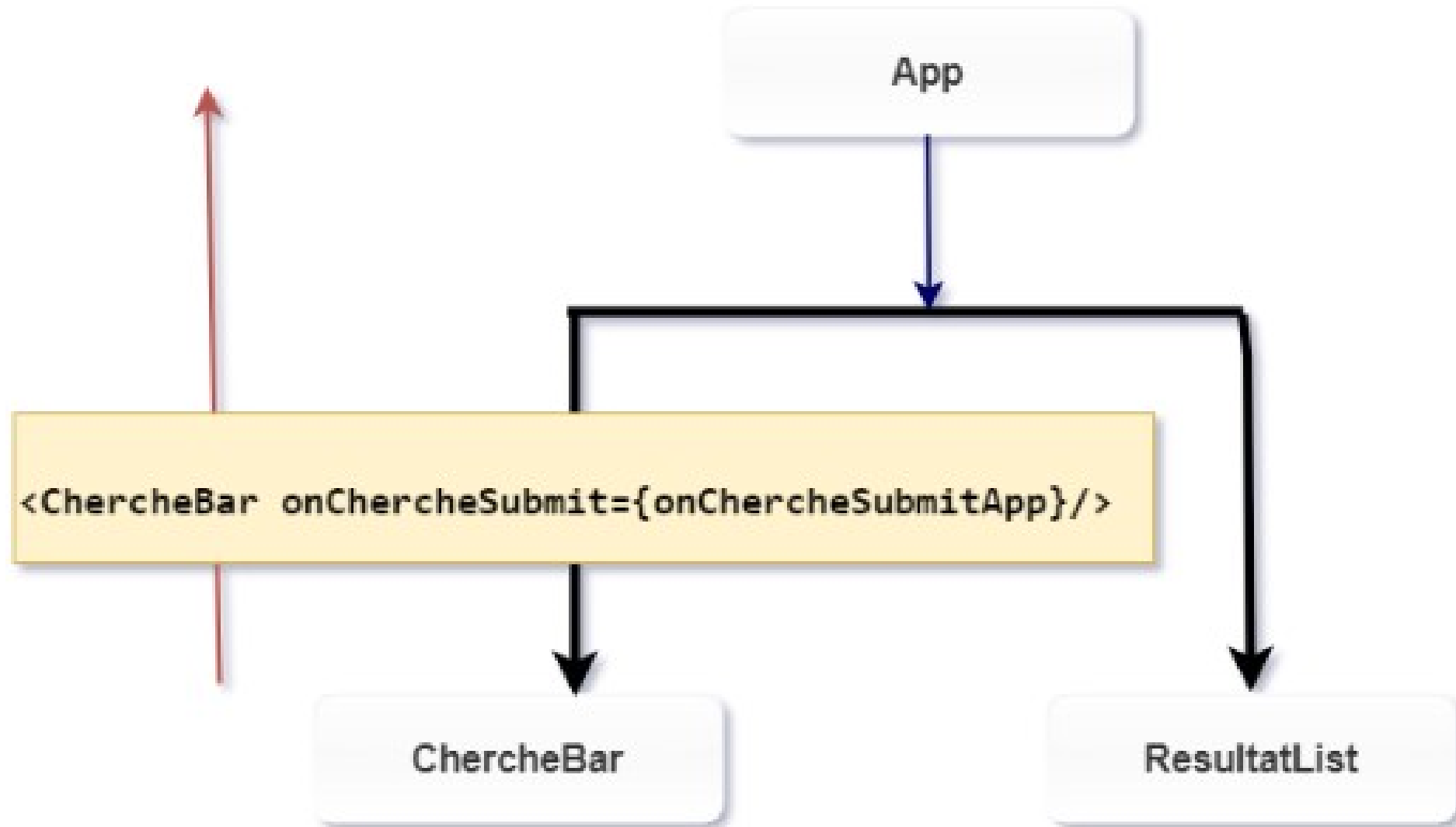


Exemple de communication inter-composants: communication du parent vers l'enfant

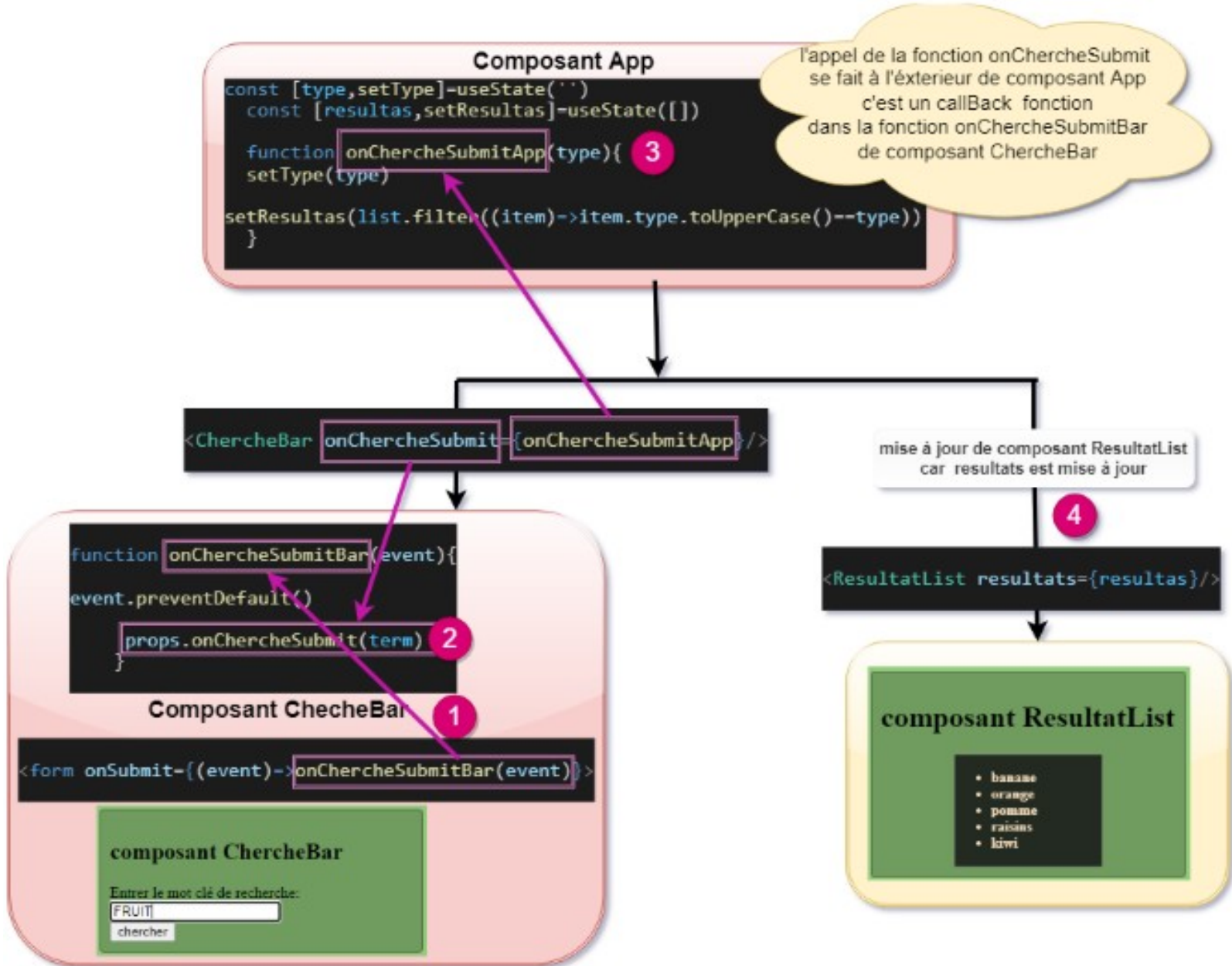
Le composant ResultatList reçoit l'information resultas via le props resultats:



Exemple de communication inter-composants: communication d'enfant vers parent



Exemple de communication inter-composants: récapitulatif



Exemple de communication inter-composants: explication du code

On a passé au composant `ChercheBar` le props `onChercheSubmit={onChercheSubmitApp}` qui contient la callback fonction `onChercheSubmitApp`.

```
<ChercheBar onChercheSubmit={onChercheSubmitApp}/>
```

```
const [resultas,setResultas]=useState([])
function onChercheSubmitApp(type){
  setType(type)
  setResultas(list.filter(item => item.type.toUpperCase()==type))
}
```

- `onChercheSubmitApp` met à jour le type par la valeur de l'argument, puis filtre la liste par le type dont la valeur est passée en argument.
- `onChercheSubmitApp` sera exécuté quand l'utilisateur soumet le formulaire. Le composant `ResultatList` est ré-rendu avec la liste résultats filtrée.

Exemple de communication inter-composants: explication du code (ChercheBar.js)

```
import { useState } from "react";
export default function ChercheBar({onChercheSubmit}) {
  const [term,setTerm]=useState('');
  function onChercheSubmitBar(event){
    event.preventDefault();
    onChercheSubmit(term);
  }
  return (
    <div className="Child">
      <form onSubmit={event => onChercheSubmitBar(event)}>
        <h2>composant ChercheBar</h2>
        <div>
          <label>Entrer le mot clé de recherche:</label>
          <input type="text" value={term}
            onChange={event =>setTerm(event.target.value.toUpperCase())} />
        </div>
        <button type="submit">chercher</button>
      </form>
    </div>
  );
}
```

Exemple de communication inter-composants: explication du code (ResultatList.js)

```
export default function ResultatList({resultats}) {  
  return (  
    <div className="Child">  
      <h1>composant ResultatList</h1>  
      {resultats.length == 0 ? (  
        <p>pas de resultats</p>  
      ) : (  
        <div className="list">  
          <ul>  
            {resultats.map(item => <li key={item.nom}>{item.nom}</li> )}  
          </ul>  
        </div>  
      )}  
    </div>  
  );  
}
```

Exemple de communication inter-composants: explication du code (récapitulatif)

